1 Evaluation Infrastructure and Implementation Details

This appendix provides detailed documentation of the case study infrastructure, submission processing pipeline, and repository organization. All source code, documentation, and specifications are available in the public repository: [GitHub] bsc-apm-case-study-task.

1.1 Repository Structure and Organization

The case study infrastructure is organized into a public repository (`bsc-apm-case-study-task`) containing all task specifications, grading infrastructure, and participant materials:

- `task/` - Task specification materials:
  - `Specification.md` - Product Requirements Document (PRD)
  - `Default-Prompt.md` - Default prompt for AI assistant conversations
  - `SCORING.md` - Detailed scoring rubric (48 points total)
- `grader/` - Grading and data processing infrastructure:
  - `src/` - Core grading modules (protocol testing, memory testing, scoring)
  - `pipeline/` - Submission processing pipeline scripts
  - `scripts/` - Utility scripts for chat parsing and result viewing
  - `fixtures/` - SSO mock HTML fixtures for testing
  - `grade` - Main grading entry point script
- `participant-guides/` - LaTeX participant documentation:
  - `participant-guide-cursor.tex` - Sub-Study A guide (Cursor)
  - `participant-guide-copilot.tex` - Sub-Study B guide (GitHub Copilot)

For complete repository documentation, see: [GitHub] bsc-apm-case-study-task/README.md

1.2 Baseline Reproducibility

All tests are run against fixed baseline commits to ensure reproducibility:

- **eclass-mcp-server:** Commit `8b4e39c` (main branch)
- **openeclass:** Commit `e8b3329` / Docker image `openeclass:test`

The grading infrastructure applies participant patches against these exact baseline versions. The baseline score (no patch applied) is **12/48 points**, representing only the Python baseline functionality.

Environment setup instructions, including Docker configuration and test data seeding, are documented in: [GitHub] bsc-apm-case-study-task/grader/SETUP.md

1.3 Test Suite Structure

The automated test suite evaluates submissions across three categories (48 points total):

- **Python Baseline (12 points)** - Tests existing functionality from main branch (11 test cases: build, protocol, login, logout, courses, authstatus)
- **Python Feature (12 points)** - Tests new `get_course_announcements` implementation (5 test cases: error handling, date filtering, output format)
- **C Replica (24 points)** - Tests complete C implementation (16 test cases: compilation, memory safety, protocol compliance, all tool functionality)

Test definitions and point allocations are specified in `grader/src/scorer.py`. Test execution logic is implemented in:

- `grader/src/protocol_tester.py` - MCP protocol and tool functionality tests
- `grader/src/memory_tester.py` - Valgrind memory leak detection
- `grader/src/build_tester.py` - Build system verification

For the detailed scoring rubric, see: [GitHub] bsc-apm-case-study-task/task/SCORING.md

For complete test specifications, see the source code documentation in `grader/src/`.

1.4 Submission Workflow and Format

This section describes the complete participant submission process: what participants do, how they ensure session isolation, and what they deliver.

1.4.1 Task Workflow

Participants complete the task in two isolated sessions, each using a different AI tool. The workflow for each session is:

1. Create an isolated Git branch from the clean baseline
2. Work on the task using the assigned AI tool
3. Generate submission artifacts (code patch and interaction logs)
4. Clean up the session branch to ensure isolation

1.4.2 Session Isolation Mechanism

To ensure clean separation between sessions and minimize learning effects from contaminating results, participants work on isolated Git branches:

1. **Session 1:** Create branch `s1_{tool}` from clean `main` branch
2. Work on task, commit changes to session branch

3. Generate patch: `git diff main s1_{tool} > {PID}_S1_{platform}_{tool}_solution.patch`
4. Export logs and save patch to safe location
5. Delete session branch: `git branch -D s1_{tool}`
6. **Session 2:** Repeat process with `s2_{tool}` branch

This workflow ensures that:

- Each session starts from an identical baseline state
- No code or context from Session 1 influences Session 2
- Patches accurately represent only the changes made during that specific session

1.4.3 Submission Format and Contents

Submissions follow a strict naming format to enable automated processing:
`{PID}_{S1|2}_{platform}_{tool}_submission.zip`

Where:

- `{PID}` - Participant ID (e.g., `P001`)
- `{S1|2}` - Session number (`S1` for first tool, `S2` for second tool)
- `{platform}` - Platform identifier: `cursor` or `copilot` (lowercase)
- `{tool}` - Tool identifier: `apm`, `spec-kit`, `cursor-plan`, or `copilot-plan` (lowercase, hyphenated)

Example: `P001_S1_cursor_apm_submission.zip`

Each submission zip file must contain exactly two items at the root level:

1. **Patch file** - `{PID}_{S1|2}_{platform}_{tool}_solution.patch`
   - Git patch file created via `git diff main s{1|2}_{tool}`
   - Contains all code changes for the session
2. **Logs folder** - `{PID}_{S1|2}_{platform}_{tool}_logs/`
   - `chats/` - Exported chat conversations (`.md` for Cursor, `.json` for Copilot)
   - `stats/` - (Cursor only) Filtered CSV usage statistics

1.5 Participant Guides and Data Collection

The case study provides platform-specific participant guides due to differences in data collection methods and tool workflows. Both guides include identical task specifications, environment setup requirements, and submission procedures. The differences reflect platform-specific data collection capabilities.

For the complete participant guides, see: [GitHub] bsc-apm-case-study-task/participant-guides

1.5.1 Sub-Study A (Cursor)

The Cursor participant guide (`participant-guide-cursor.tex`) includes:

- Instructions for exporting chat conversations as `.md` files
- Instructions for exporting CSV usage statistics from Cursor dashboard (see Section 1.5.1)
- Python script workflow for filtering CSV by session start time
- Organization of logs into `chats/` and `stats/` subfolders
- Requirement to use Cursor Composer 1 model for entire session

Cursor Usage Statistics Collection:

Cursor participants export CSV files containing all AI requests across their account, then filter by session start time:

1. Export CSV from Cursor Dashboard ([Link] cursor.com/dashboard)
2. Note session start time at beginning of work session
3. Run filtering script: `python3 filter_csv_by_datetime.py usage-events.csv {date} {time} {timezone}`
4. Filtered CSV contains only requests from session start time onward
5. Save filtered CSV as `{PID}_{S1|2}_cursor_{tool}_requests.csv` in `stats/` subfolder

This method enables precise session isolation when participants may have other Cursor activity outside the study session.

1.5.2 Sub-Study B (GitHub Copilot)

The Copilot participant guide (`participant-guide-copilot.tex`) includes:

- Instructions for exporting chat conversations as `.json` files via VS Code Command Palette
- No CSV statistics export (Copilot does not provide session-based usage dashboards)
- Organization of logs into `chats/` subfolder only
- Hybrid model strategy: Premium model (Claude Sonnet 4) for Setup/Manager, budget model for Implementation

1.6 Data Privacy and Anonymization

To protect participant privacy while enabling reproducibility, all chat exports are processed through anonymization and sanitization procedures before storage. This process is performed during the parsing stage of the grading pipeline.

1.6.1 Metadata Preservation

Each chat conversation is assigned a unique identifier using cryptographic hashing for anonymous cataloging:

- **Method:** SHA256 hash of concatenated user messages
- **Format:** First 16 hexadecimal characters of the hash
- **Purpose:** Enables unique identification of chats without revealing participant identity or conversation content

The chat ID is computed before sanitization and preserved in plaintext metadata, allowing cataloging and deduplication without decryption. Along with the chat ID, the following metadata is preserved in plaintext:

- **Model IDs:** Distinct language models used in the conversation
- **Request Count:** Total number of AI requests in the chat
- **Timestamp:** Sanitization timestamp
- **Original Size:** Size of original payload in bytes

This metadata enables dataset organization, integrity validation, and basic analysis without requiring access to encrypted content.

1.6.2 Sanitization Process

Chat exports are sanitized using reversible encryption to protect sensitive information such as file paths, command histories, and timestamps:

- **Encryption Method:** HMAC-SHA256 XOR cipher
  - Keystream generation via HMAC-SHA256 expansion with random salt
  - XOR encryption of serialized chat payload
  - Base64 encoding of salt and ciphertext
- **Storage Format:** `ENC:<salt_b64>:<ciphertext_b64>`
- **Reversibility:** Decryption using the shared secret produces data bitwise-identical to the original

For implementation details, see: [GitHub] bsc-apm-case-study-task/grader/scripts

1.7 Grading Pipeline Architecture

The grading infrastructure processes submissions through a multi-stage pipeline with checkpoint-based error recovery. Each stage is logged in a manifest file (see Section 1.8) with checksums and timestamps for audit trails and reproducibility.

1.7.1 Pipeline Stages

The submission processing workflow consists of six discrete stages:

1. **Validation** - Validates zip structure and naming format, extracts submission to workspace root. Script: `pipeline/validate_submission.py`
2. **Grading** - Runs automated black-box test suite against Python and C implementations. Outputs results to JSON and TXT files. Script: `pipeline/grade_submission.py`
3. **Parsing** - Extracts interaction metrics from chat exports and sanitizes chat logs using reversible encryption (see Section 1.6 for details). Script: `pipeline/parse_submission.py`
4. **Combination** - Merges grading results with extracted interaction metrics into unified participant result files. Script: `pipeline/combine_participant.py`
5. **Dataset Aggregation** - Appends processed submission data to platform-level CSV datasets. Script: `pipeline/add_to_dataset.py`
6. **Integrity Validation** - Validates dataset integrity against manifest checksums and timestamps. Script: `pipeline/validate_dataset.py`

For detailed pipeline documentation, see: [GitHub] bsc-apm-case-study-task/grader/PIPELINE.md

1.8 Pipeline Manifest System

The pipeline uses a manifest-based checkpointing system for traceability and error recovery. Each platform maintains a manifest at:
`submissions_sanitized/{platform}/pipeline_manifest.json`

1.8.1 Manifest Structure

Manifest entries use key format: `{PID}_{S1|2}_{Platform}_{Tool}`
Each entry tracks:

- **Validation** - Zip checksum, extraction timestamp, validation status
- **Grading** - Test results, scores, grading timestamp
- **Parsing** - Metrics extraction status, sanitization checksums
- **Combination** - Merged result file checksum
- **Dataset** - CSV row checksum, dataset append timestamp
- **Validation** - Integrity check results

The processing history for any submission can be queried via:
[GitHub] bsc-apm-case-study-task/grader/pipeline/inspect_submission.py.

1.9 Additional Resources

- **Manifest Implementation Details:** [GitHub] bsc-apm-case-study-task/grader/pipeline/manife
- **Task Specification:** [GitHub] bsc-apm-case-study-task/task
- **Grading Infrastructure:** [GitHub] bsc-apm-case-study-task/grader
- **Participant Guides:** [GitHub] bsc-apm-case-study-task/participant-guides
- **Pipeline Documentation:** [GitHub] bsc-apm-case-study-task/grader/PIPELINE.md
- **Setup Instructions:** [GitHub] bsc-apm-case-study-task/grader/SETUP.md